

GDPRFS

A FUSE-based GDPR-Compliant File System

Real-time enforcement of data-subject rights at the filesystem layer

Wei-En Hsieh: Semester Project, ETH Zürich

Supervisors: François Hublet, David Basin

Outline

1. Motivation
2. System setup and architecture
3. Communication flows
4. GDPR articles
5. Evaluation results
6. Conclusion

I/ Motivation

Problem

Problem:

- Prior work mainly enforces GDPR inside applications, e.g. MiniTwit-style web apps, but not on disk.
- In practice, GDPR-relevant personal data is often stored in files on disk.
- Once on disk, these files can be opened by any program without consent or purpose checks.
- No file-system-level GDPR enforcement tool.

Solution

- Goal: build a GDPR-compliant file system
- GDPRFS built on FUSE (Filesystem in Userspace)
- Any program reading a file automatically sees redacted content if consent is missing.
- The enforcement logic is a single formal policy (MFOTL) as in previous work.

```
from fuse import Fuse

class MyFS(Fuse):
    """A minimal GDPR-compliant FUSE filesystem."""
```

Core fragment of GDPR

Article	Right / Principle
5	Lawfulness, purpose limitation, accuracy
6	Lawful basis (consent)
9	Special categories (health, genetic, biometric, ...)
15	Right of access
16	Right to rectification
17	Right to erasure
30	Records of processing

Every file read, write, rename, or delete is checked against these articles in real time.

II/ System setup and architecture

1) Setup

Entities

- one internal user: access to files
- one or many external users / data subjects (DS): whose personal data lives in the files

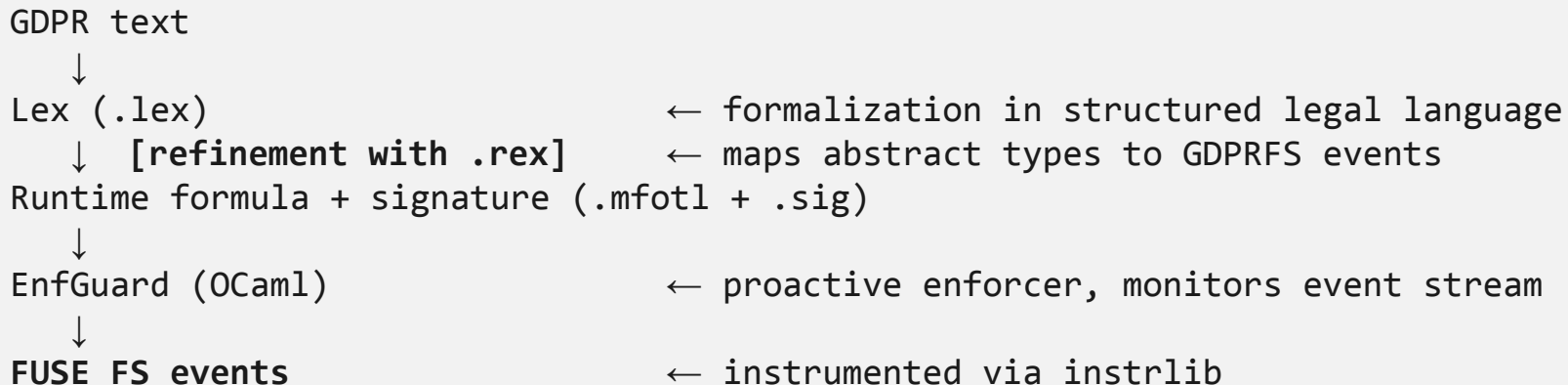
Files

- Supported file formats: TXT, PDF, CSV
- Mount point: /tmp/mnt (FUSE FS)

External services

- Able to deploy web applications for internal user (on computer) and data subjects (online)
- Access to a LLM API (GPT-5-nano)

2) Enforcement methodology from Lex

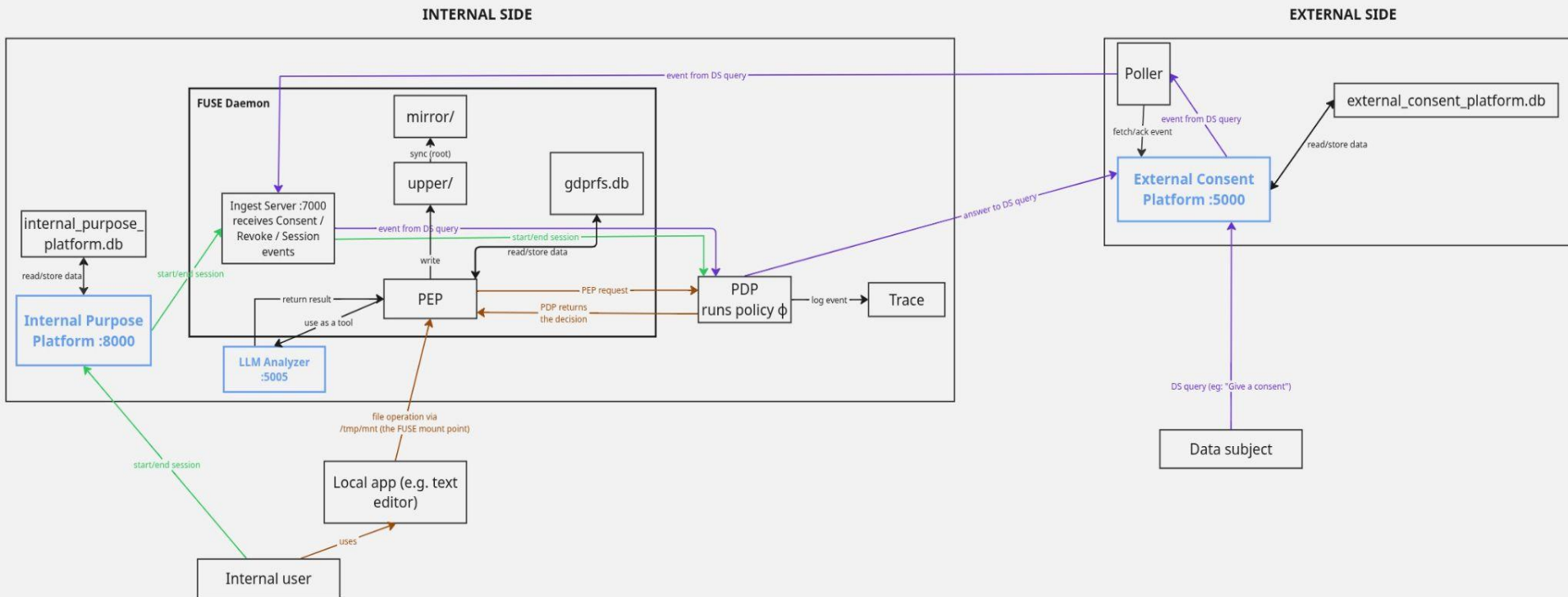


[1] F. Hublet et al., *Scaling Up Proactive Enforcement*, CAV'25.

[2] F. Hublet et al., *Lex: Turning Laws into Enforceable Security Policies*, working draft.

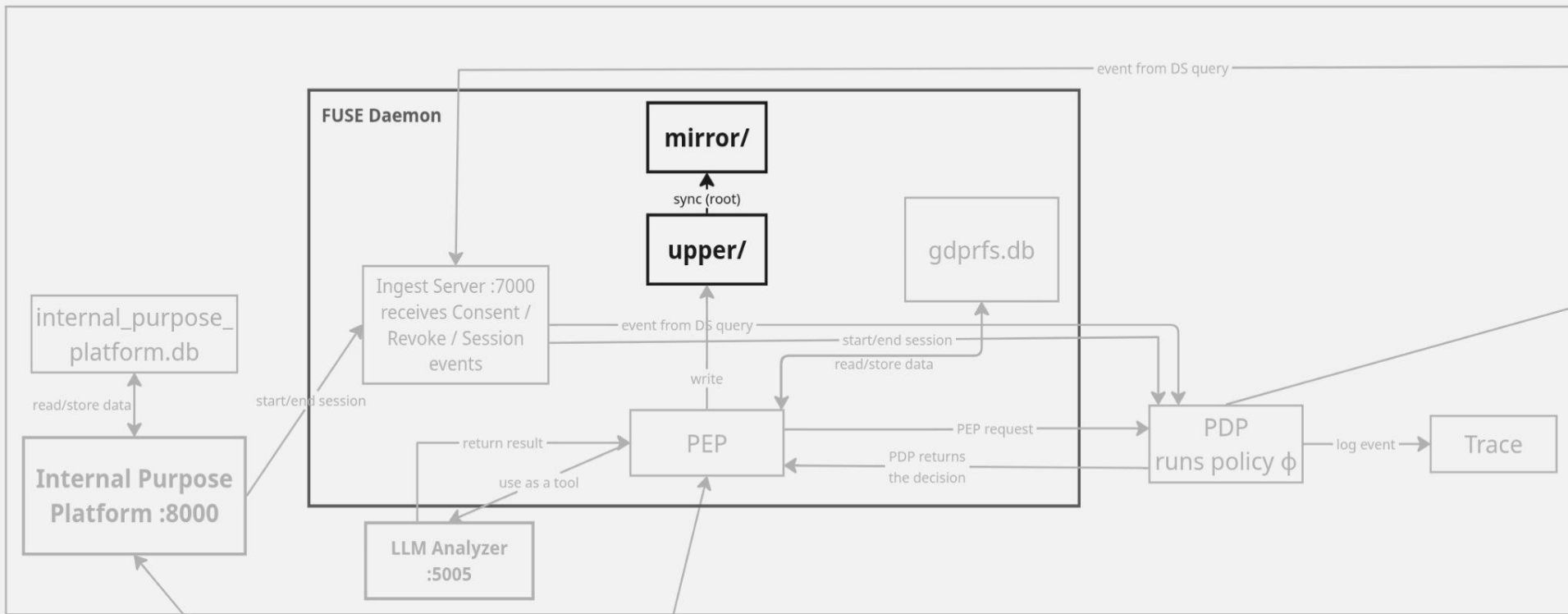
3) Architecture

GDPRFS Architecture Diagram



4) Two-layer storage

INTERNAL SIDE



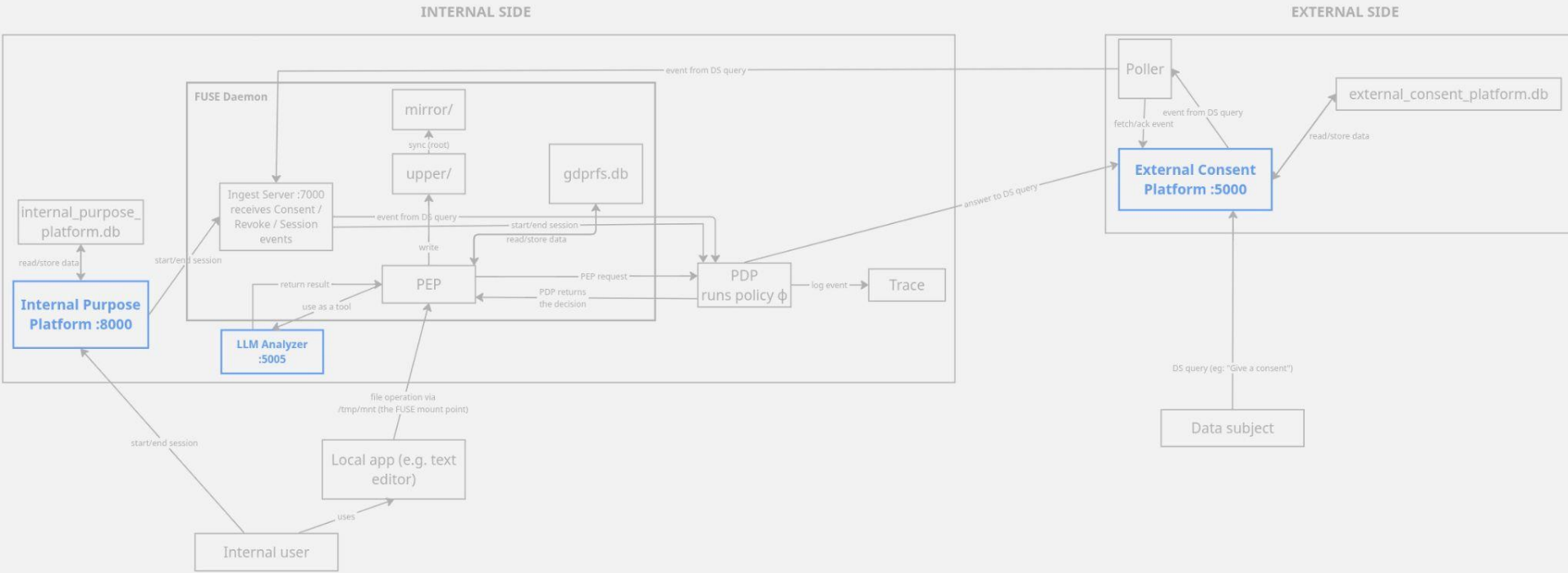
4) Two-layer storage contd.

Layer	Path	Access	Purpose
Upper	/var/lib/gdprfs/upper/	User-writable via FUSE	Working copy: every op goes here
Mirror	/var/lib/gdprfs/mirror/	Root-only, mode 0o700	Tamper-proof audit backup

- Users see /tmp/mnt (the FUSE mountpoint).
- Every successful write or rename is synced to mirror by the daemon (running as root).
- Users cannot reach mirror directly; only the FUSE daemon touches it.
- Deletes (via erasure) remove from both layers + DB.

5) Core components

GDPRFS Architecture Diagram



5) Core components contd.

Component	Port	Role
External Consent Platform	5000	Data-subject-facing portal: DS grants/revokes consent, requests access, rectification, erasure.
Internal Purpose Platform	8000	Internal-user-facing portal: start/stop a session, choose a purpose, see merge alerts when PII is detected.
LLM Analyzer	5005	Used for PII + special-category data detection; called on every write. GPT-5-nano via Pydantic-AI.

6) PII detection: 4-tier hierarchy

Hierarchy	Tier	Example
1	.gdprowner (config file)	jdoe: secretproject/** : all files under secretproject/ → jdoe
2	Folder name matches a Person	folder fhublet/ → François Hublet
3	Filename contains a Person name	jdoe_report.txt → John Doe
4	File content (with LLM)	LLM finds "John Doe" in body → jdoe

- Tier 1 exists because naming conventions don't always encode ownership (secret projects, codenames).
- Tiers 2–3 are cheap string matches.
- Tier 4 is the fallback when path-based tiers give nothing.

III/ Communication flows

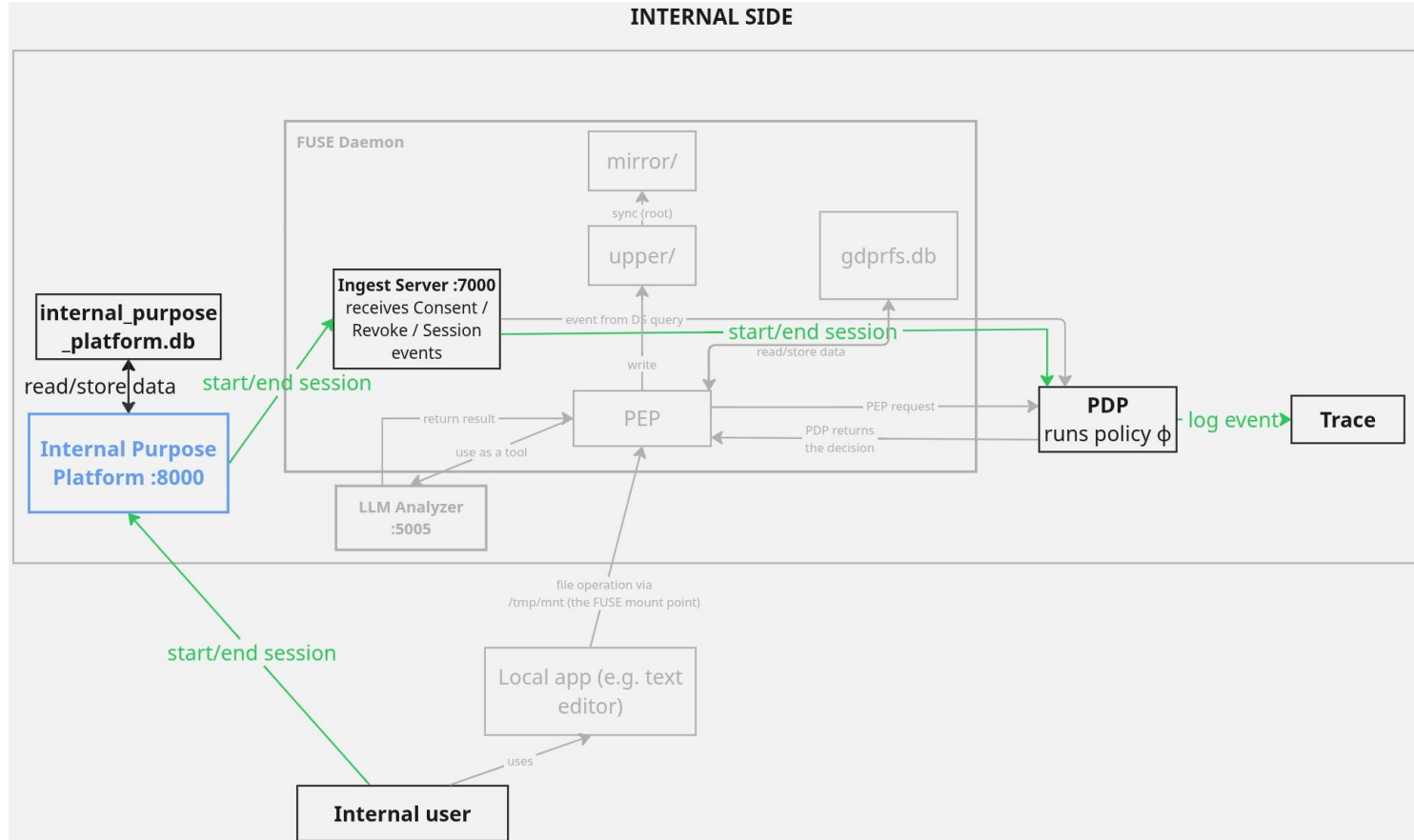
Read path: redaction

1. Active session? No => return "REDACTED".
2. For each data subject linked to the file: query consent platform.
3. If Missing consent => redact according to the file format

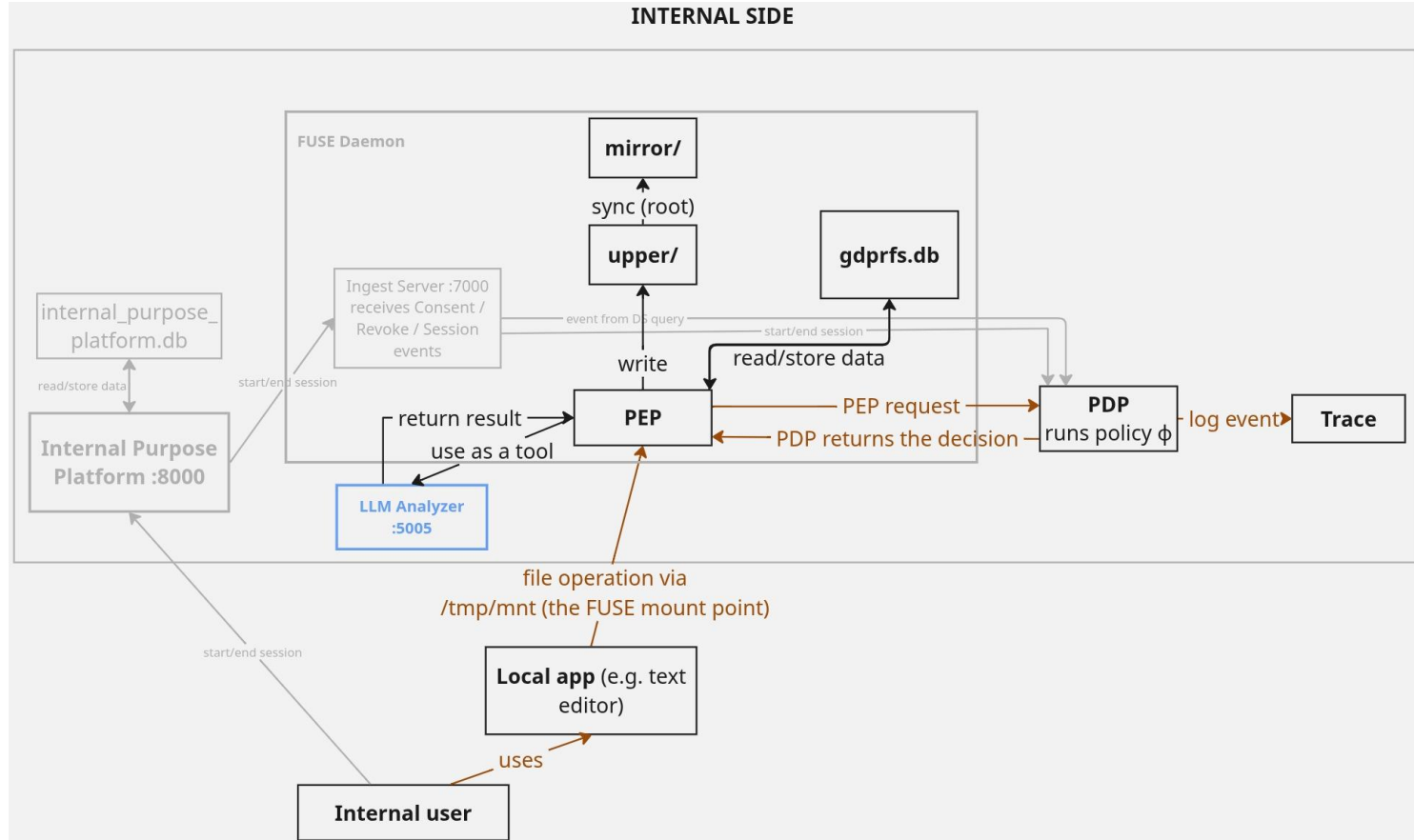
File format	Granularity	Replacement
TXT	Whole file	Content → "REDACTED"
PDF	Per page	Blank "REDACTED" page template
CSV	Per row	Row cells → "REDACTED"

Redaction is computed on the fly at read time. A revoke is visible on the very next read.

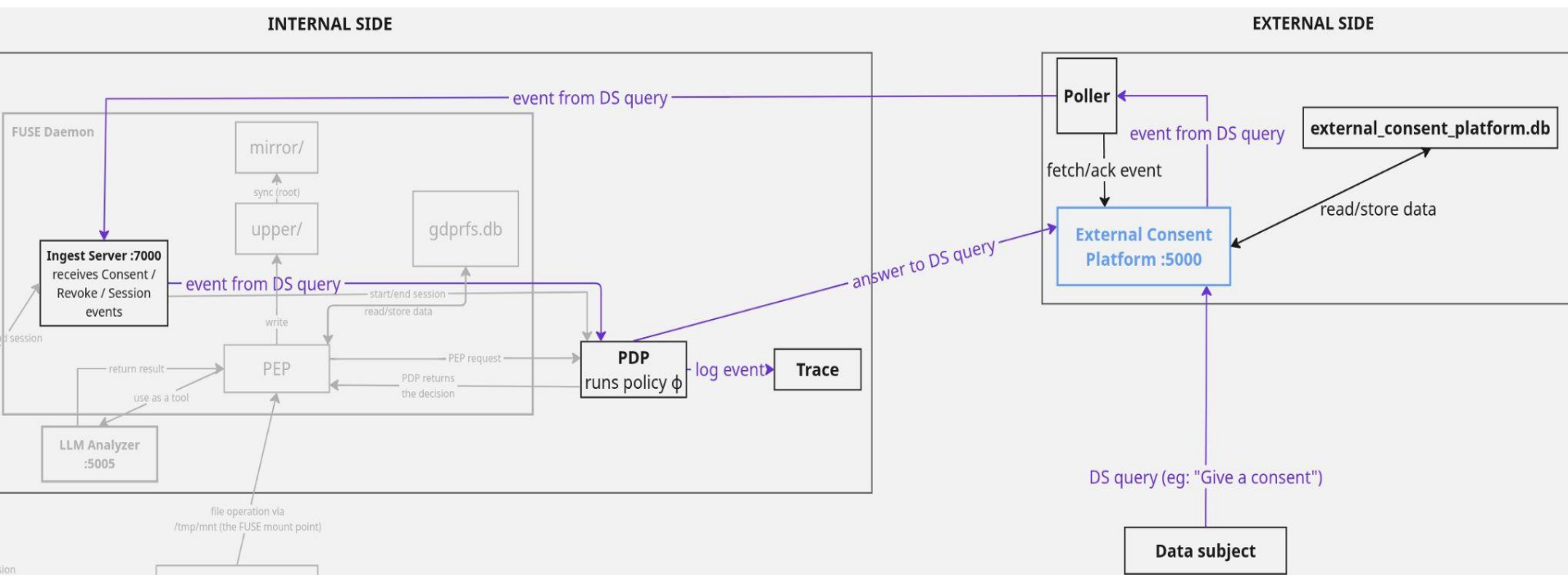
Flow 1: Start or end session



Flow 2: Write to file



Flow 3: Consent from data subject



IV/ GDPR articles

Articles 5 & 6

Formalizing the Law in Lex

Article 5: Principles relating to processing of personal data

Personal data shall be:

- (a) processed lawfully, fairly and in a transparent manner in relation to the data subject;
- Article 5(1) GDPR

```
suppressable event DataProcessing
```

```
    """Data {d} is processed by processor {p}  
    on behalf of controller {c} as part of  
    data processing activity {a}"""
```

```
p : entity
```

```
c : entity
```

```
a : activity
```

```
d : data
```

```
article "5"
```

```
paragraph "1"
```

```
point "a"
```

```
rule
```

```
    whenever
```

```
        DataProcessing(p, c, a, d)
```

```
        PersonalData(d, ds)
```

```
    oblige
```

```
        EXISTS b. IsLawful(a, b)
```

```
        IsFair(a)
```

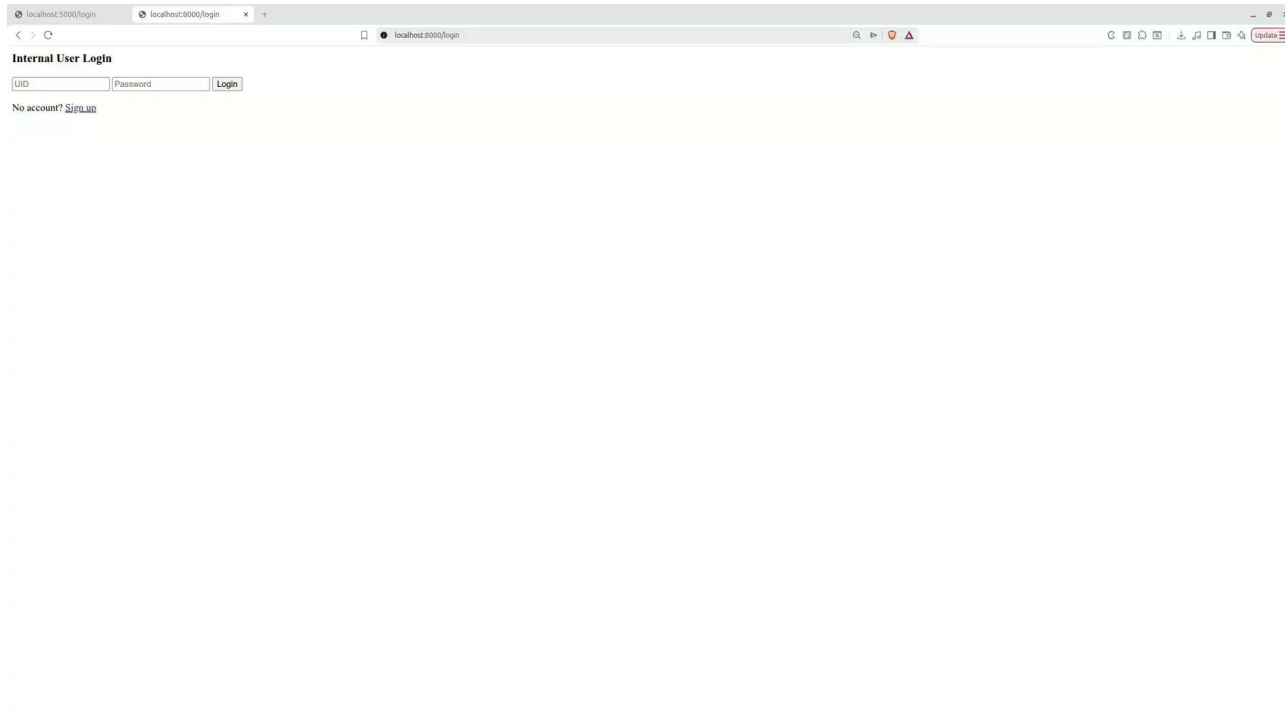
```
        IsTransparent(a, ds)
```

```
transparently enforceable suppressing DataProcessing
```


Refining Legal Specifications in Lex

```
rule "r_DataProcessing_Use"  
  whenever  
    Use(d, _)  
  refine  
    DataProcessing("GDPRFS", "GDPRFS", "Use", d)  
rule "r_DataProcessing_Write"  
  whenever  
    Write(d, _)  
  refine  
    DataProcessing("GDPRFS", "GDPRFS", "Write", d)
```

Art 5 & 6 workflow example



The screenshot shows a web browser window with two tabs. The active tab is titled 'localhost:5000/login'. The address bar shows 'localhost:5000/login'. The page content is as follows:

Internal User Login

UID Password

No account? [Sign up](#)

Article 9

Formalizing the Law in Lex

Article 9: Processing of special categories of personal data

Processing of special categories of personal data, such as racial or ethnic origin, political opinions, religious beliefs, genetic, biometric, health, sex-life, or sexual-orientation data, shall be prohibited.

— Article 9(1) GDPR

```
observable predicate IsSpecialData
```

```
"""The data {d} reveals information of special data category {sp}.  
Special data categories include personal data revealing racial  
or ethnic origin, political opinions, religious or philosophical  
beliefs, trade union membership, the processing of genetic data,  
biometric data for the purpose of uniquely identifying a natural  
person, data concerning health or data concerning a natural person's  
sex life or sexual orientation"""
```

```
d : data
```

```
sp : special_data_category
```

Formalizing the Law in Lex contd.

Article 9: Processing of special categories of personal data

Processing of special categories of personal data, such as racial or ethnic origin, political opinions, religious beliefs, genetic, biometric, health, sex-life, or sexual-orientation data, shall be prohibited.

— Article 9(1) GDPR

```
article "9"  
paragraph "1"
```

```
rule  
| whenever  
|   IsSpecialData(d, sp)  
|   PersonalData(d, ds)  
| oblige  
|   NOT DataProcessing(pr, c, a, d)  
transparently enforceable causing effects
```

```
article "9"  
paragraph "2"  
point "a"
```

```
rule "special_data_consent_exception"  
| whenever  
|   IsSpecialData(d, sp)  
|   PersonalData(d, ds)  
|   DataProcessing(pr, c, a, d)  
|   ONCE GiveSpecialConsent(ds, p, c, sp)  
| except  
|   paragraph "1"
```

Refining Legal Specifications in Lex

```
rule "r_IsSpecialData"  
  whenever  
    SpecialData(d, spCat)  
  refine  
    IsSpecialData(d, spCat)
```

Article 15

Formalizing the Law in Lex

Article 15: Right of access by the data subject

The data subject shall have the right to obtain from the controller confirmation as to whether or not personal data concerning him or her are being processed, and [...] access to the personal data and [...]

(b) the categories of personal data concerned;

— Article 15(1) GDPR

```
observable predicate IsAccessRequest
  """Request {rq} is an access request"""
  rq : request
```


Formalizing the Law in Lex contd.

Article 15: Right of access by the data subject

The data subject shall have the right to obtain from the controller confirmation as to whether or not personal data concerning him or her are being processed, and [...] access to the personal data and [...]

(b) the categories of personal data concerned;

— Article 15(1) GDPR

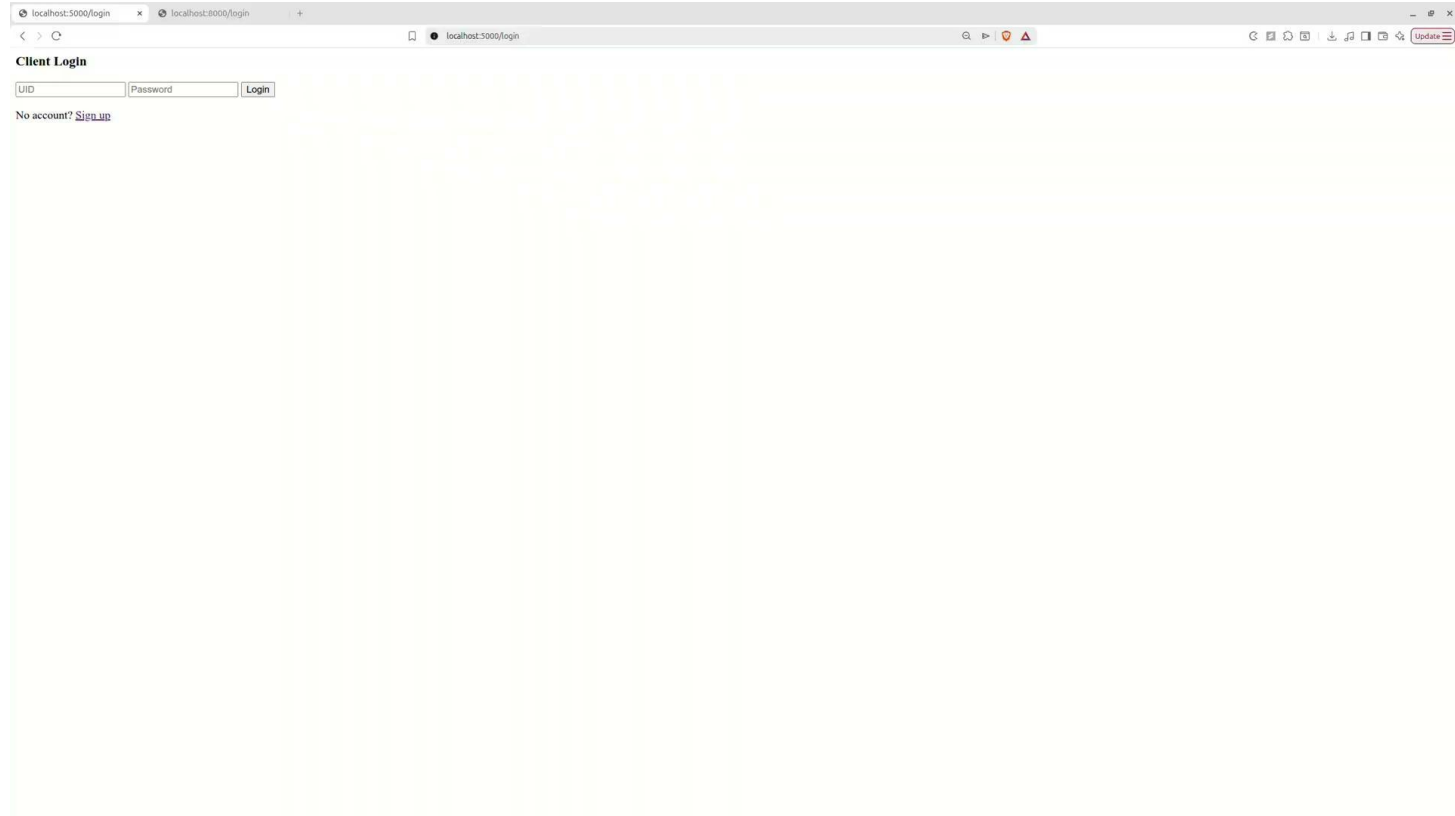
```
article "15"  
paragraph "1"  
point "b"
```

```
rule  
| whenever  
|   ONCE (Request(ds, rq, c) AND IsAccessRequest(rq))  
|   RequestResponse(ds, rq, rs)  
|   EXISTS pr, a, d. (ONCE DataProcessing(pr, c, a, d) AND PersonalData(d, ds) AND HasCategory(d, cat))  
| oblige  
|   EXISTS de. Contains(rs, de) AND IsCategory(de, cat)  
transparently enforceable causing effects
```

Refining Legal Specifications in Lex

```
rule "r_IsAccessRequest"  
  whenever  
    RequestAccess(ds)  
  refine  
    Request(ds, "access", "GDPRFS")  
    IsAccessRequest("access")
```

Art 15 workflow example



The screenshot shows a web browser window with two tabs: 'localhost:5000/login' and 'localhost:8000/login'. The active tab is 'localhost:5000/login'. The page title is 'Client Login'. The form contains two input fields: 'UID' and 'Password', followed by a 'Login' button. Below the form, there is a link for 'Sign up'.

Client Login

UID Password Login

No account? [Sign up](#)

Article 16

Formalizing the Law in Lex

Article 16: Right to rectification

The data subject shall have the right to obtain [...] the rectification of inaccurate personal data [...] and to have incomplete personal data completed, including by means of a supplementary statement.

— Article 16 GDPR

```
article "16"

rule "rectification_obligation"
  whenever
    | IsObligatedToRectify(c, d, d')
  oblige
    | (NOT UndueDataDelay(d)) UNTIL Rectify(d, d')
  transparently enforceable causing effects

rule "rectification_request_inaccuracy"
  whenever
    | ONCE (Request(ds, rq, c) AND IsRectificationRequest(rq, d, d'))
    | RequestResponse(ds, rq, rs)
    | EXISTS pr, a, d. (ONCE (DataProcessing(pr, c, a, d) AND PersonalData(d, ds)))
    | HasInaccuracy(d)
  constitute
    | IsObligatedToRectify(c, d, d')
```

Refining Legal Specifications in Lex

```
rule "r_RectificationRequest"  
  whenever  
    RequestRectification(ds, d, d')  
  refine  
    Request(ds, "rectification", "GDPRFS")  
    IsRectificationRequest("rectification", d, d')  
    HasInaccuracy(d)
```

Article 17

Formalizing the Law in Lex

Article 17: Right to erasure ('right to be forgotten')

The data subject shall have the right to [...] erasure of personal data concerning him or her without undue delay and the controller shall have the obligation to erase personal data without undue delay [...]

(a) the personal data are no longer necessary [...]

(b) the data subject withdraws consent [...] and where there is no other legal ground for the processing;

— Article 17(1) GDPR

```
article "17"  
paragraph "1"  
  
rule  
| whenever  
|   IsObligatedToDelete(c, d)  
| oblige  
|   NOT UndueDataDelay(d) UNTIL Delete(d)  
transparently enforceable causing effects
```


Formalizing the Law in Lex contd.

Article 17: Right to erasure ('right to be forgotten')

The data subject shall have the right to [...] erasure of personal data concerning him or her without undue delay and the controller shall have the obligation to erase personal data without undue delay [...]

(a) the personal data are no longer necessary [...]

(b) the data subject withdraws consent [...] and where there is no other legal ground for the processing;

— Article 17(1) GDPR

```
article "17"  
  paragraph "1"  
    point "a"
```

```
rule  
  whenever  
    (ONCE (EXISTS rq. Request(ds, rq, c) AND IsErasureRequest(rq, d)) AND RequestResponse(ds, rq, rs)) OR DataReview(c, d)  
    NOT (EXISTS pr, a, d, p. (ONCE (DataProcessing(pr, c, a, d) AND PersonalData(d, ds) AND HasPurpose(a, p))) AND IsNecessary(d, p))  
  constitute  
    IsObligatedToDelete(c, d)
```

Formalizing the Law in Lex contd.

Article 17: Right to erasure ('right to be forgotten')

The data subject shall have the right to [...] erasure of personal data concerning him or her without undue delay and the controller shall have the obligation to erase personal data without undue delay [...]

(a) the personal data are no longer necessary [...]

(b) the data subject withdraws consent [...] and where there is no other legal ground for the processing;

— Article 17(1) GDPR

```
article "17"  
paragraph "1"  
point "b"
```

```
rule  
  whenever  
    (ONCE (EXISTS rq. Request(ds, rq, c) AND IsErasureRequest(rq, d)) AND RequestResponse(ds, rq, rs)) OR DataReview(c, d)  
    NOT (EXISTS pr, a, d. (NOT WithdrawConsent(ds, p, c)) SINCE (DataProcessing(pr, c, a, d) AND PersonalData(d, ds)  
    AND (IsLawful(a, "6(1)(a)") OR IsLawful(a, "9(2)(a)"))) AND NOT (EXISTS ba. IsLawful(a, ba)))  
  constitute  
    IsObligatedToDelete(c, d)
```

Refining Legal Specifications in Lex

```
rule "r_ErasureRequest"  
  whenever  
    RequestErasure(ds, d)  
  refine  
    Request(ds, "erasure", "GDPRFS")  
    IsErasureRequest("erasure", d)
```

```
rule "r_WithdrawConsent"  
  whenever  
    Revoke(ds, p)  
  refine  
    WithdrawConsent(ds, p, "GDPRFS")
```

Article 30

Formalizing the Law in Lex

Article 30: Records of processing activities

Each controller and, where applicable, the controller's representative, shall maintain a record of processing activities under its responsibility.

(a) the name and contact details of the controller [...];

— Article 30(1) GDPR

```
article "30"  
  paragraph "1"  
    point "a"  
  
rule "controller_record"  
  whenever  
    | DataProcessing(pr, c, a, d)  
  oblige  
    | Record(pr, c, a, "Controller", string_of_entity(c))  
transparently enforceable causing effects
```

Formalizing the Law in Lex contd.

Article 30: Records of processing activities

The obligations referred to in paragraphs 1 and 2 shall not apply to an enterprise or an organisation employing fewer than 250 persons unless [...] the processing includes special categories of data as referred to in Article 9(1) [...].

— Article 30(5) GDPR

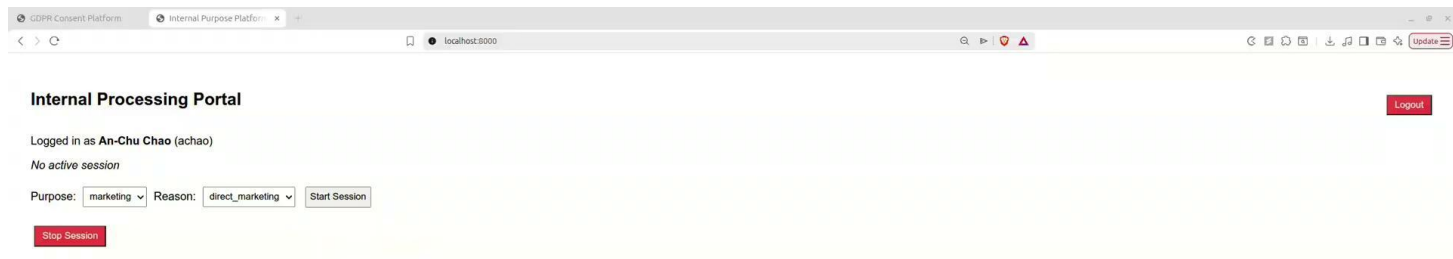
```
article "30"  
paragraph "5"  
  
rule "SME_exemption"  
  whenever  
    | DataProcessing(pr, c, a, d)  
    | IsSME(c)  
    | IsSME(pr)  
  except  
    | paragraph "1"  
    | paragraph "2"
```

```
article "30"  
paragraph "5"  
rule  
  whenever  
    | IsRiskyProcessing(a)  
    | NOT IsOccasionalProcessing(a)  
    | EXISTS d', sp. (DataProcessing(pr, c, a, d')  
    |   AND IsSpecialData(d', sp)  
    |   OR RelatesToCriminalConvictionsOrOffences(d'))  
  )  
  except  
    | rule "SME_exemption"
```

Refining Legal Specifications in Lex

```
rule "r_IsRiskyProcessing"  
  """Processing special category data is considered risky for SMEs (Art 30(5) counter-exception)."""  
  whenever  
  |   SpecialData(d, spCat)  
  refine  
  |   IsRiskyProcessing("Use")
```

Art 30 workflow example



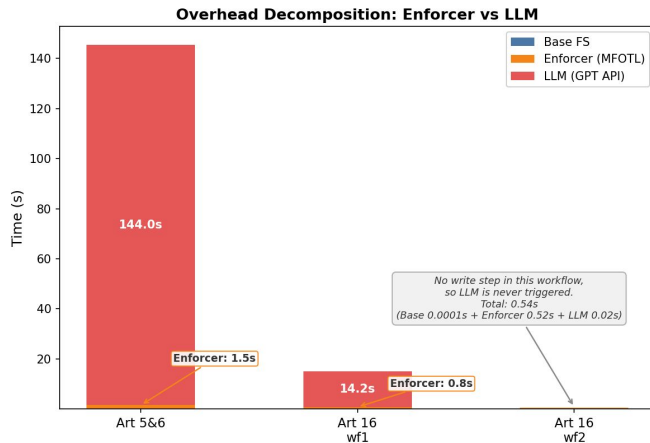
V/ Evaluation results

1) Benchmark methodology

Mode	Enforcer	LLM	Measures
baseline	X	X	Only Base FS cost
gdpr_no_llm	✓	X	Enforcer overhead + Base FS cost
gdpr_with_llm	✓	✓	Full system (Enforcer overhead + LLM overhead + Base FS cost)

- One or two workflow script per article in benchmark/.
- Typical workflow: session start → consent → **write** → read → revoke → read → erase → session stop.

2) Benchmark results



Benchmark	baseline	+ enforcer	+ enforcer + LLM	Enforcer	LLM
Art 5 & 6 (4 writes)	0.009 s	1.53 s	145.57 s	1.5 s	144 s
Art 16 wf1 (1 write)	0.0002 s	0.76 s	15.00 s	0.76 s	14.2 s
Art 16 wf2 (0 writes)	0.0001 s	0.52 s	0.54 s	0.52 s	0.02 s

Enforcer adds sub-second up to ~1.5 s. LLM dominates: Art16 wf2 has no write so no LLM used.

VI/ Conclusion

Conclusion, limitations, future work

Conclusion

- built a GDPR-compliant file system: any program reading a file gets consent-checked and redacted automatically.
- Seven articles enforced by a single formal MFOTL policy
- Covers the full lifecycle: **detect** PII and special data, **enforce** (consent, session, special categories), **respond** (Art 15/16/17), **record** (Art 30).

Known limitations

- LLM is the bottleneck (e.g. 144 s on write-heavy workflows) and costs real money per call (OpenAI API tokens).
- TXT / PDF / CSV formats: no DOCX, images, binary blobs yet.

Future work

- Privacy concern: feeding **sensitive** data to LLM => use local distilled model to replace GPT for the write event.
- Other articles of the GDPR law.

Thank you: questions?

A1: Art 9 workflow example



A2: Performance: what's inside the 144 s?

4 writes $\times$ ~ 35 s each $\rightarrow 140$ s + ~ 4 s FS baseline + 1.5 s enforcer.

Per write: HTTP round-trip (~ 200 ms) + GPT-5-nano (~ 30 s on multi-page PDFs) + event emission + DB write.

GPT latency grows with document length; short CSVs ~ 2 s.

Enforcer itself: sub-second regardless of content size (formula complexity drives cost).

A3: Cost of GPT API calls

- GPT-5-nano: ~\$0.001–\$0.01 per file, depending on length.
- Write-heavy workflow (4 mid-size PDFs): ~\$0.01–\$0.04.
- At scale (1000 writes/day): ~\$10–\$40/day, dominated by document length.
- Replaceable by local model: no architectural dependency on OpenAI.

A4: LLM reliability: false positives / negatives

False negatives (missed PII)

Tiers 1–3 run first : deterministic, covers clear cases (named folders/files, .gdprowner).

LLM is the fallback. Merge-alert UI asks a human to confirm ambiguous detections.

Residual risk is real : any LLM-based DLP carries it. No perfection claim.

False positives (bogus name detected)

Person stored with uid = NULL, registered = False.

No account → can't grant/revoke → treated conservatively downstream. No UI burden on DS.

Explainability: Pydantic-AI returns structured output with per-hit reasoning, logged alongside detection.

A5: GDPR coverage: what's missing

Article	Status
Art 7 (consent form validity)	UI concern, out of scope
Art 12–14 (transparency notices)	UI concern, out of scope
Art 20 (data portability)	Partially: Art 15 ZIP is a machine-readable export
Art 22 (automated decision-making)	future work
Art 25 (privacy by design)	Argued architecturally, not enforced
Art 32 (security of processing)	Enforced via OS permissions; crypto is future
Art 33–34 (breach notification)	Out of scope

Seven articles with direct FS implications prioritized. The rest are UI-level or organizational.

A6: Concurrency, scalability, multi-tenant

Today: single FUSE daemon, single SQLite DB, one tested internal user (achao).

What scales already

- Metadata lookups (SQLite with indices on `abs_path`, `uid`).

- LLM calls are per-write and embarrassingly parallel.

What does not

- Session table has no per-file locking : concurrent writes by two users would race.

- SQLite is a single-writer bottleneck.

Production path: PostgreSQL, per-file locking, one daemon per mount, shared policy engine.

A7: Backups, snapshots, replication

The hard question.

A snapshot of upper/mirror to off-site backup leaves the consent enforcement loop : whoever restores it sees plaintext.

Our position

In-system backups (the mirror) are covered.

Off-system backups require the same enforcement stack at restore, or encryption keyed to consent state.

Art 17 erasure explicitly does not reach off-site backups in current implementation.

A8: Security model & threat analysis

In-scope threats

Unprivileged user bypassing consent via normal FS APIs → blocked by FUSE interception.

Consent platform tampering → MFOTL policy rejects contradictory events.

Apps reading files without consent → served redacted content transparently.

Out-of-scope (documented assumptions)

Compromised root : root can bypass FUSE; we trust the OS.

Side channels (file size, timing) : redaction preserves these.

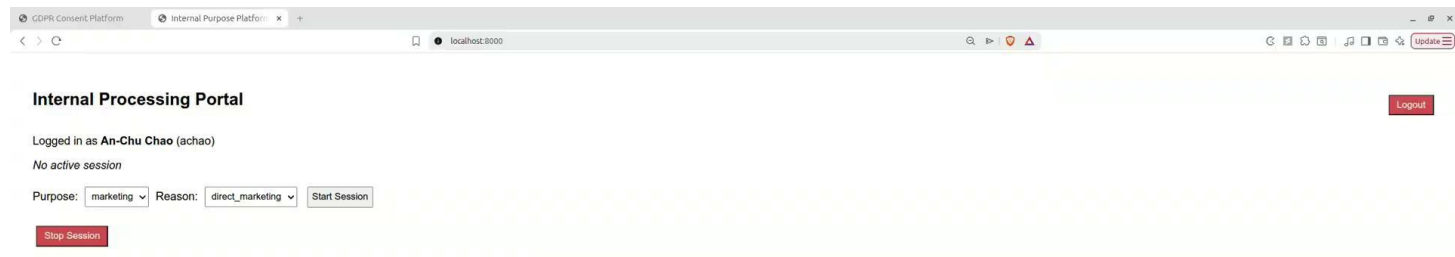
Cryptographic tamper-evidence of logs : not yet signed.

Mirror protection: mode 0o700 on /var/lib/gdprfs/mirror/. Only daemon (root) writes.

A9: What if a detected person never registers?

- LLM creates a Person row with uid = NULL, registered = False (myfs.py:248 _person_effective_uid).
- Deterministic pseudo-uid (first[:1] + last) for cross-file consistency.
- Cannot grant/revoke consent → currently default-allow (assumes incidental mentions).
- Trade-off: default-deny would break common cases; default-allow risks leaking. Configurable in future.

Art 16 workflow example



Art 17 workflow example

